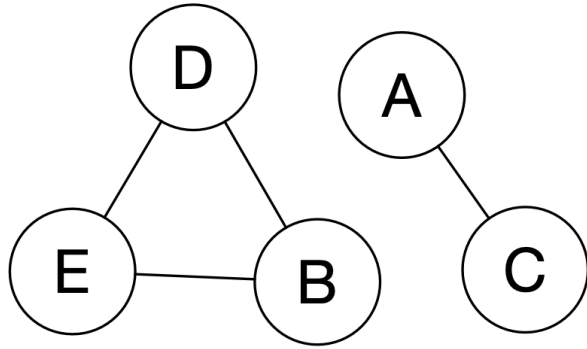


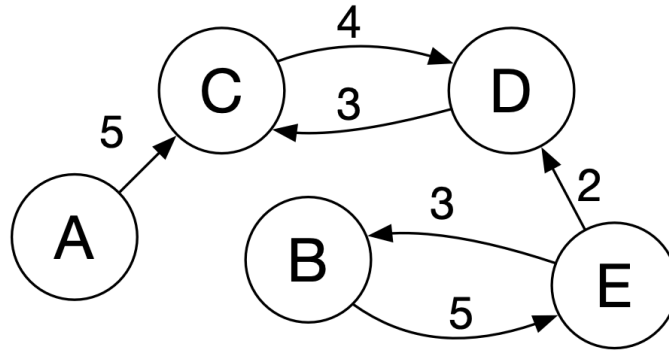
CS 3510 Algorithms: Graph Representations and DFS

Aaron Hillegass
Georgia Tech

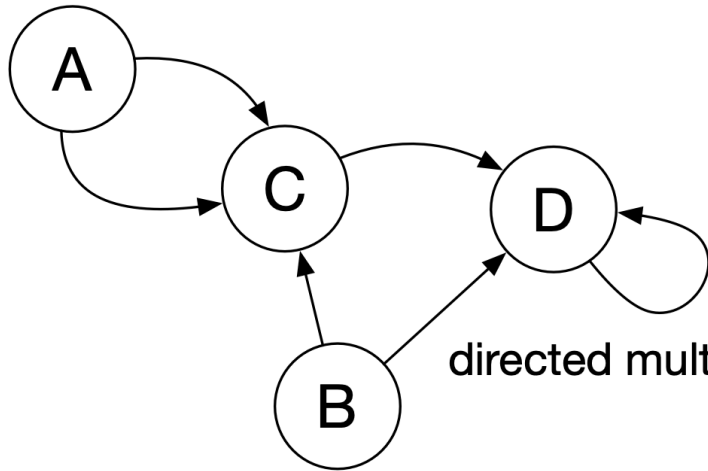
simple undirected



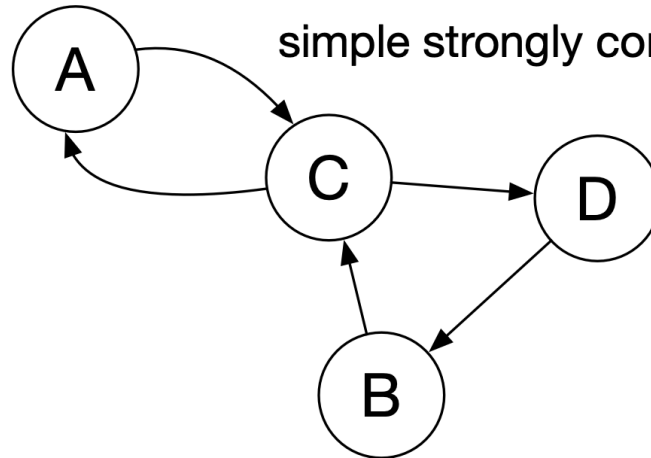
simple weighted directed



directed multigraph



simple strongly connected



Undirected Graphs

V (the vertices) are a set: $\{v_1, \dots, v_n\}$

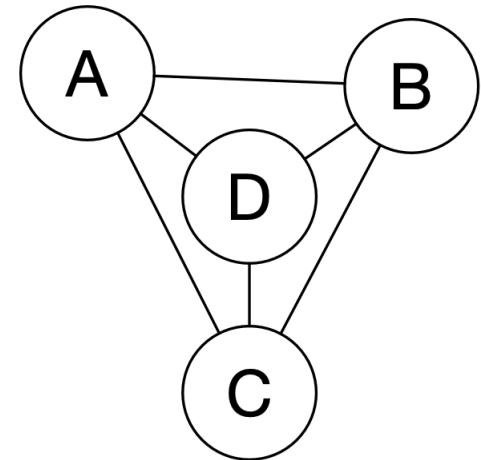
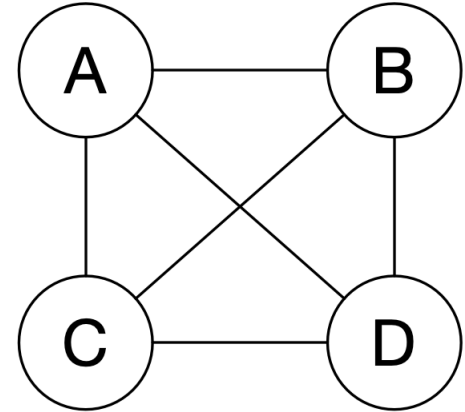
E (the edges) are a set of pairs of vertices: $\{\{u, v\} \mid u, v \in V\}$

G (the graph) is the pair (V, E)

A *path* p on G a list $[v_i \mid v_i \in V]$ such that $\{v_i, v_{i+1}\} \in E$

Two vertices u and v are *connected* if there is a path from u to v .

The graph is *connected* if every pair of vertices in V are connected.



Directed Graphs

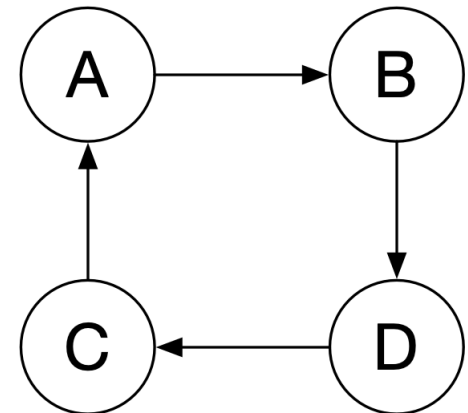
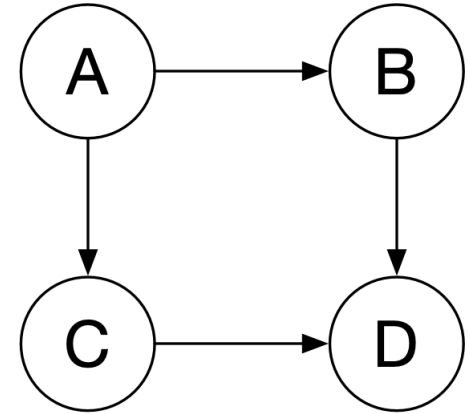
E (the edges) are a subset of $V \times V$ ((u, v) not allowed if $u = v$)

A path p from u to w a list $[v_i \mid v_i \in V]$ of length n such that $(v_i, v_{i+1}) \in E$ for all $i < n$.

u and v are *strongly connected* if there is a path from u to v and from v to u .

The graph is *strong connected* if every pair of vertices in V are *strongly connected*.

(What is a *weakly connected* graph? If you replaced the directed edges with undirected edges, the resulting undirected graph is connected.))



Representations: Adjacency Matrix

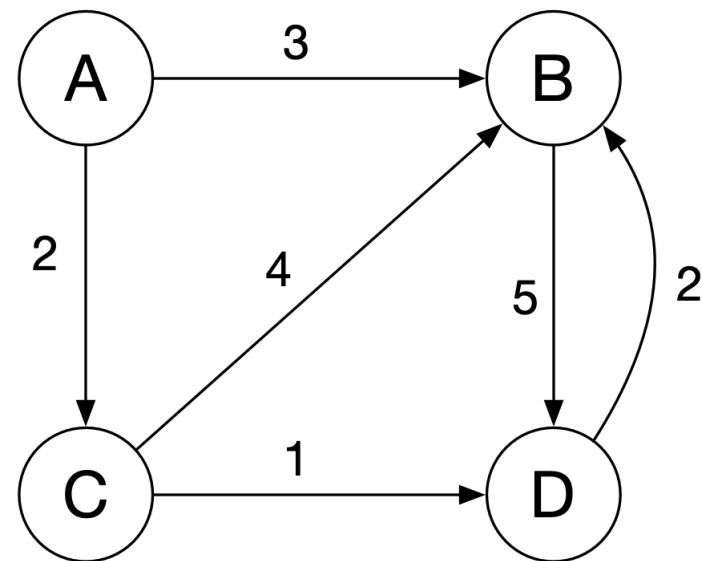
$$\begin{pmatrix} 0 & 3 & 2 & 0 \\ 0 & 0 & 0 & 5 \\ 0 & 4 & 0 & 1 \\ 0 & 2 & 0 & 0 \end{pmatrix}$$

Size: $|V|^2$

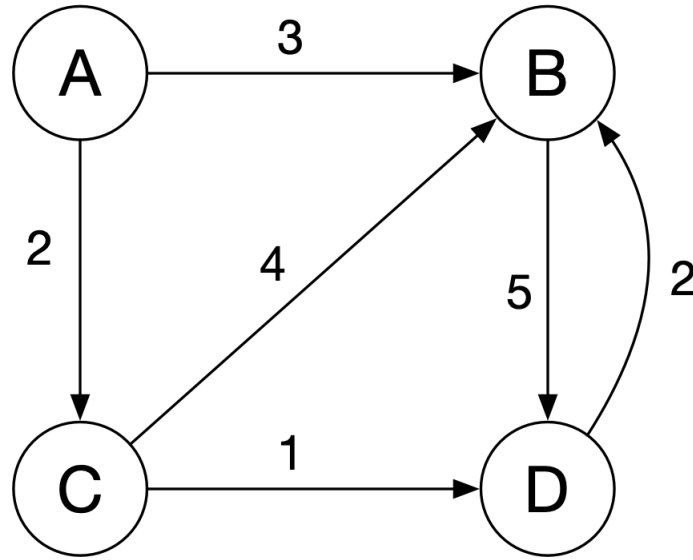
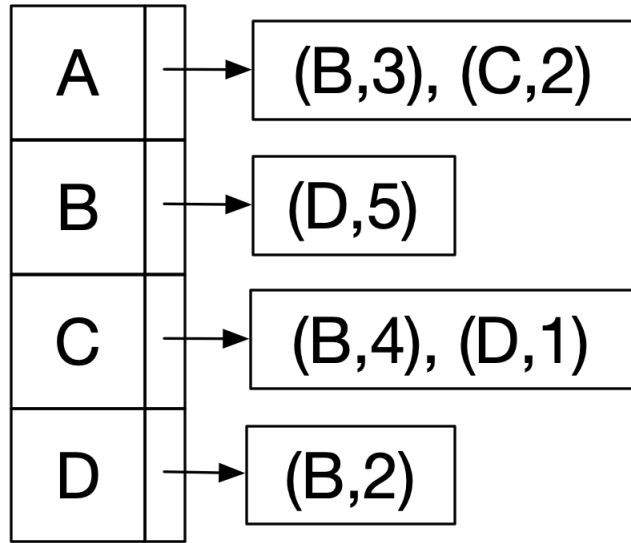
Good for dense graphs; dense graphs are uncommon.

Undirected graph? Symmetric matrix.

Unweighted graph? 0 or 1.



Representations: Adjacency List

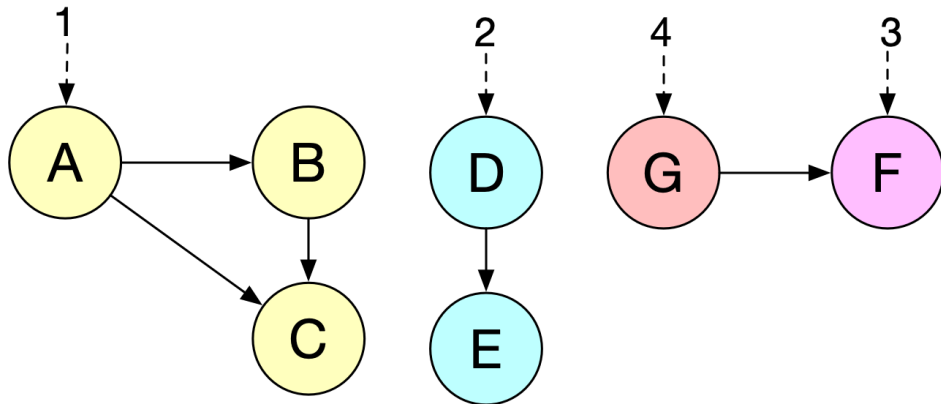


Size: $|V| + |E|$

Good for sparse graphs

```
1 define explore( $G, v$ , visited):  
2   | visited[ $v$ ] := true  
3   | for each edge  $(v, u) \in E$ :  
4   |   | if not visited[ $u$ ]:  
5   |   |   | explore( $G, u$ , visited)
```

```
1 define dfs( $G$ ):  
2   | for  $v \in V$ :  
3   |   | visited[ $v$ ] := false  
4   | for  $v \in V$ :  
5   |   | if not visited[ $v$ ]:  
6   |   |   | explore( $G, v$ , visited)
```



If G is undirected, explore will be called once for every connected component.

Previsit and Postvisit Ordering

```

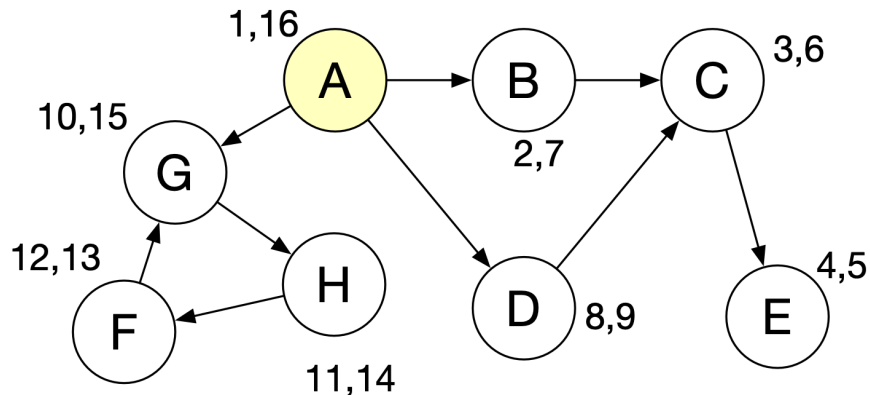
1 define explore( $G, v, \text{pre}, \text{post}, \text{clock}$ ):
2    $\text{pre}[v] := \text{counter.next}()$ 
3   for each edge  $(v, u) \in E$ :
4     if  $u \notin \text{pre}$ :
5       | explore( $G, u, \text{pre}, \text{post}, \text{clock}$ )
6    $\text{post}[v] := \text{counter.next}()$ 

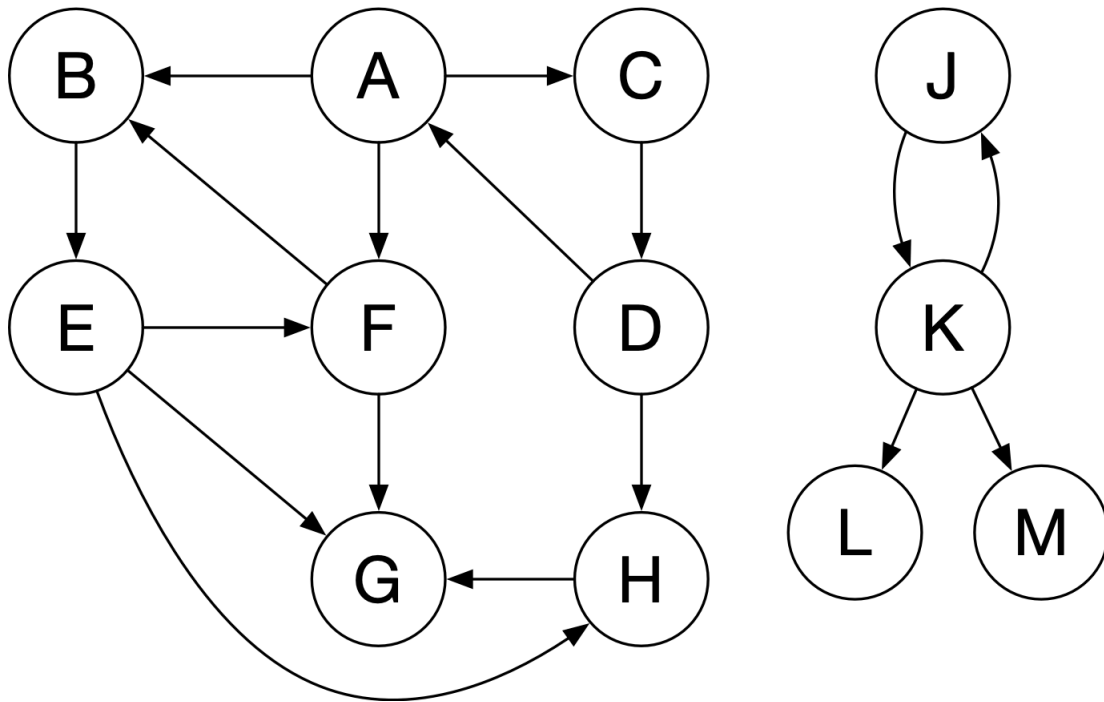
```

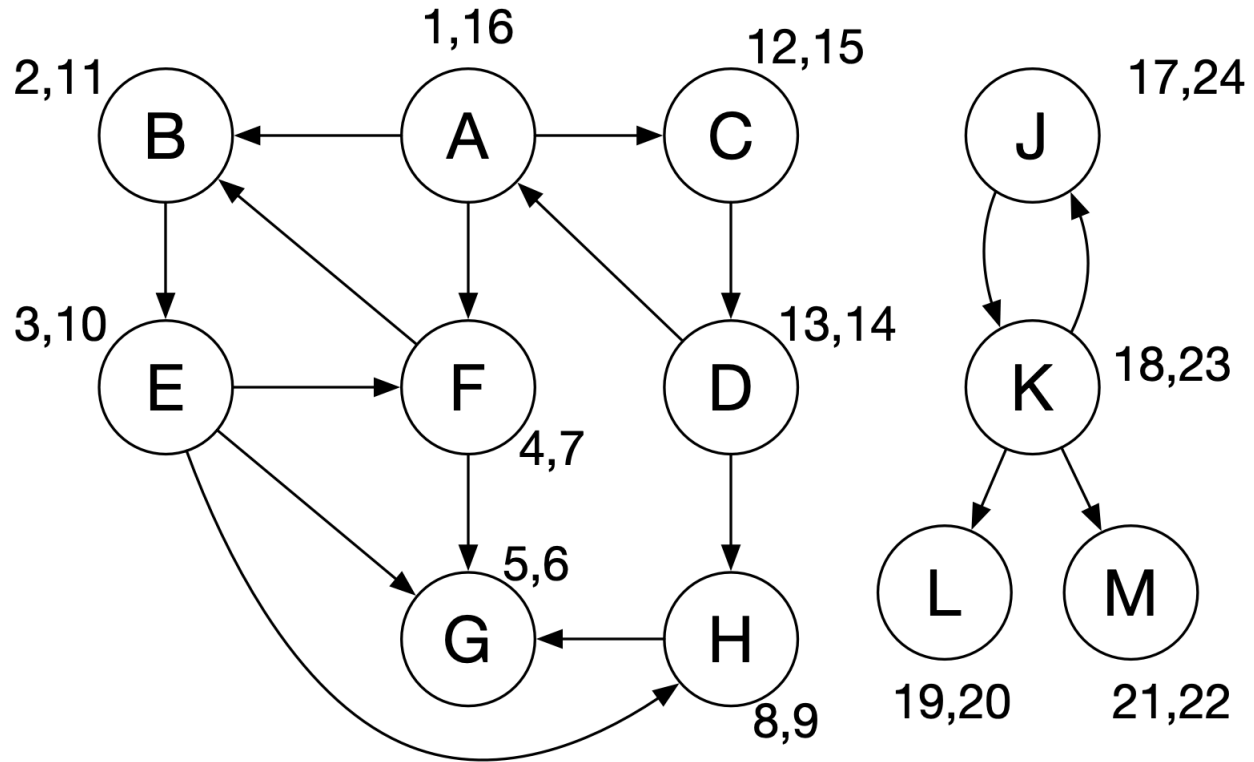
```

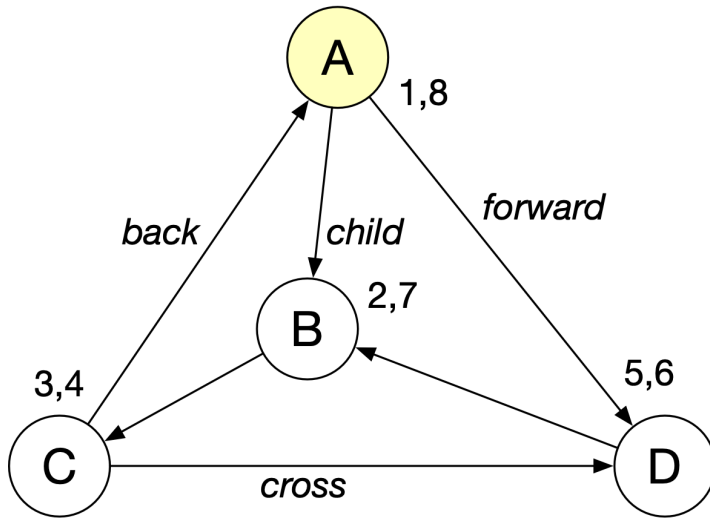
1 define dfs( $G$ ):
2   Create pre, post tables
3   Initialize counter to zero
4   for  $v \in V$ :
5     if  $v \notin \text{pre}$ :
6       | explore( $G, v, \text{pre}, \text{post}, \text{counter}$ )

```









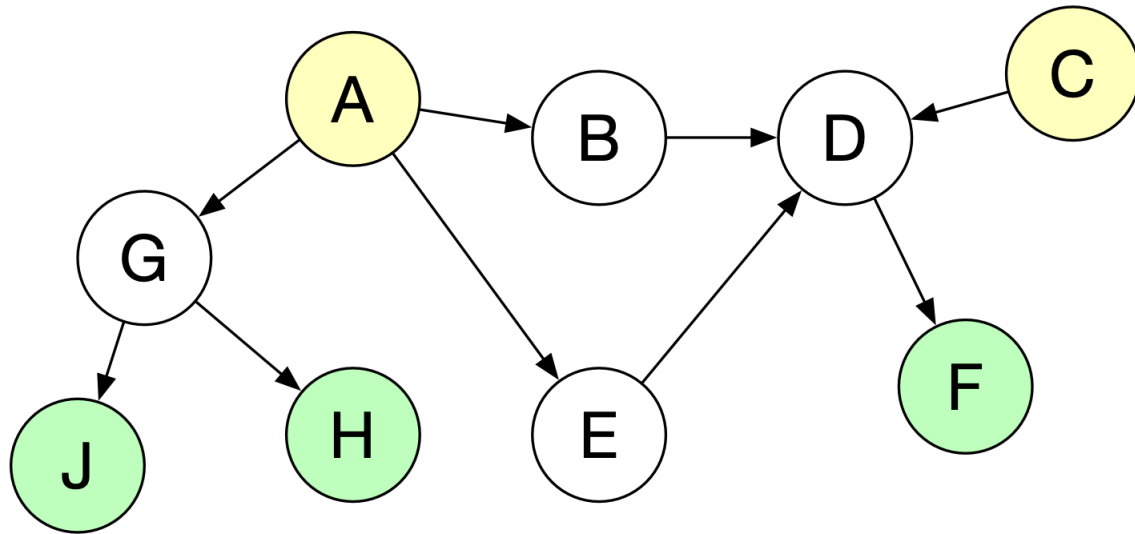
Forward, child: $\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$

Back: $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$

Cross: $\text{pre}(v) < \text{post}(v) < \text{pre}(u) < \text{post}(u)$

Directed Acyclic Graphs

13/28



Every DAG has at least one *source* and one *sink*.

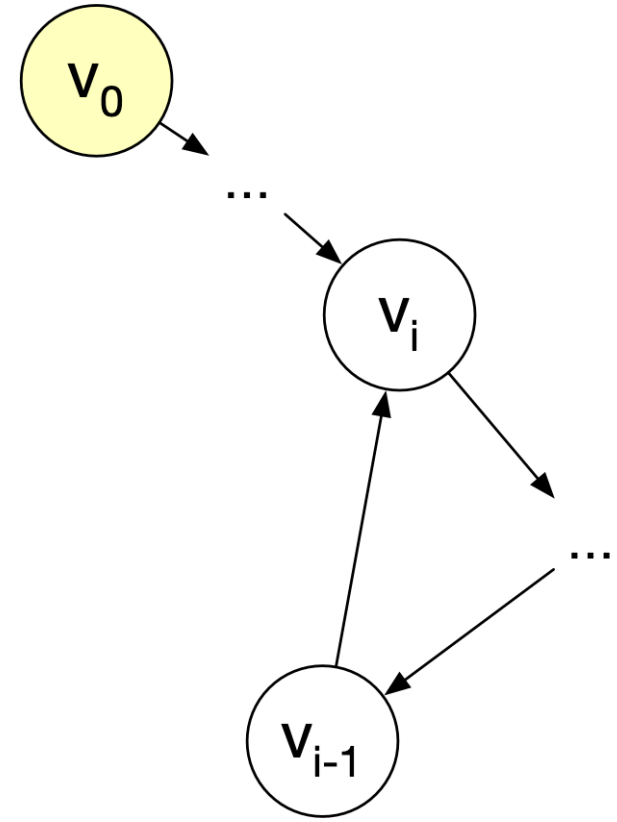
Can we use DFS to figure out if a graph has a cycle?

Is it a DAG?

DFS finds no back edge if and only if graph is acyclic.

Back edge implies a cycle seems obvious.

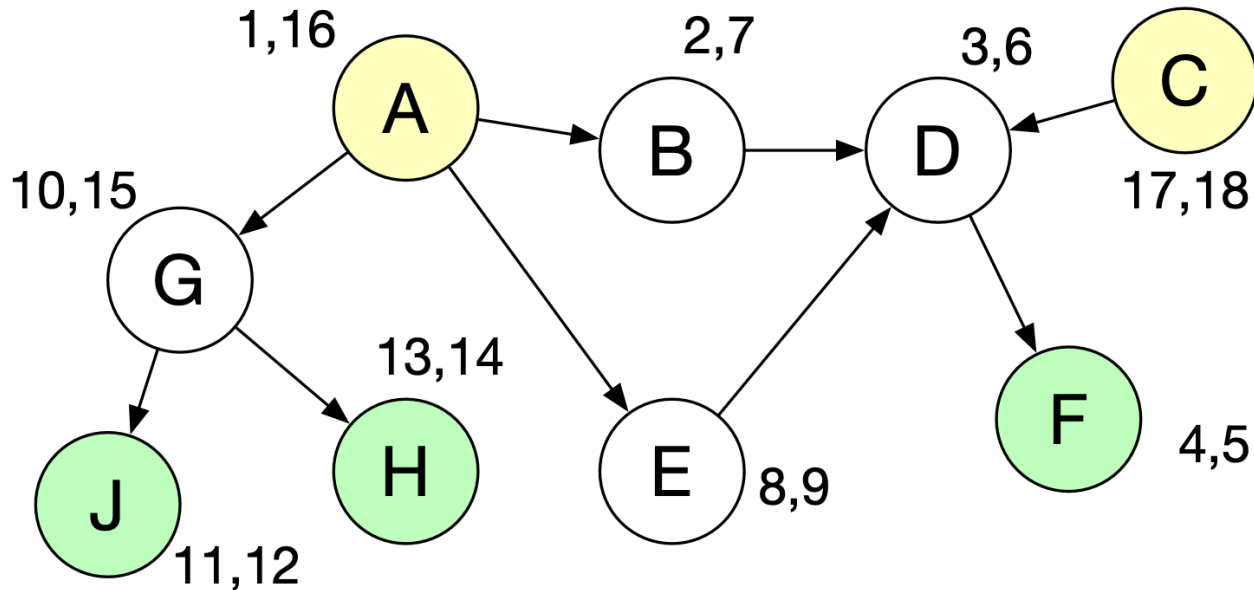
Suppose there is a cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_0$. Let v_i be the first vertex in the cycle found by DFS. All others in cycle are reachable from v_i , including v_{i-1} . There is an edge from v_{i-1} to v_i – it is a back edge.



In a DAG, every edge leads to a vertex with a lower post number.

Vertex with lowest post number must be a sink.

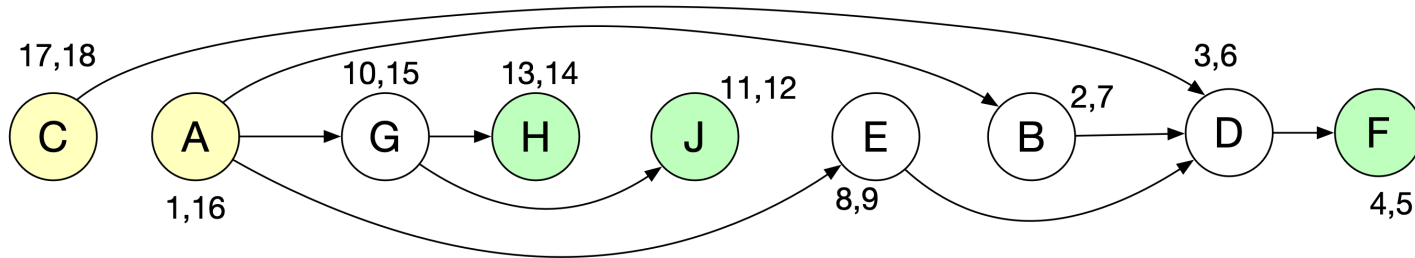
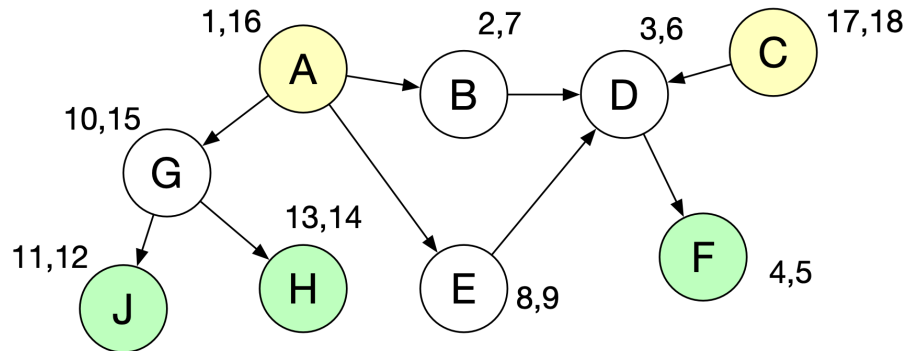
Vertex with highest post number must be a source.



Topological Sort

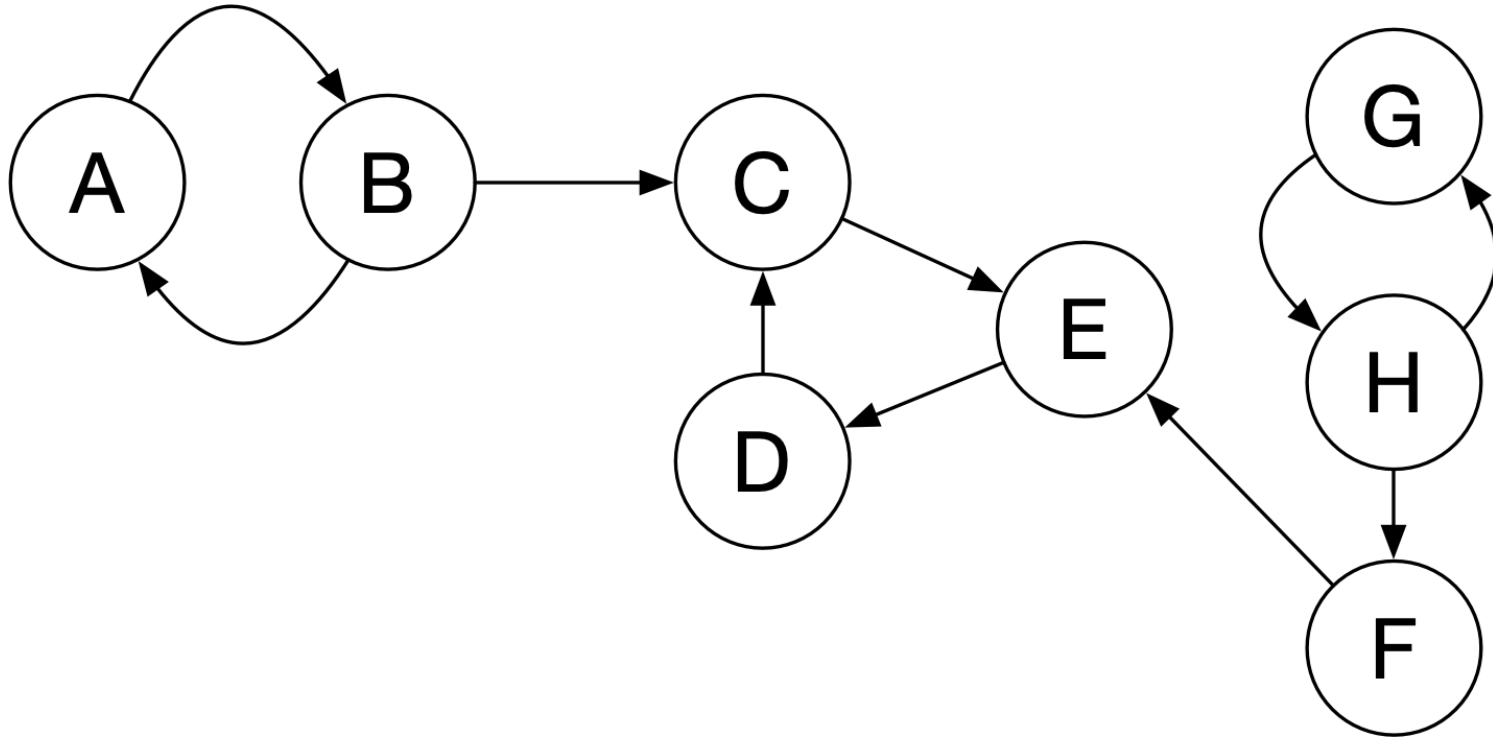
List vertices such that no edges go left.

Smallest post number comes last.



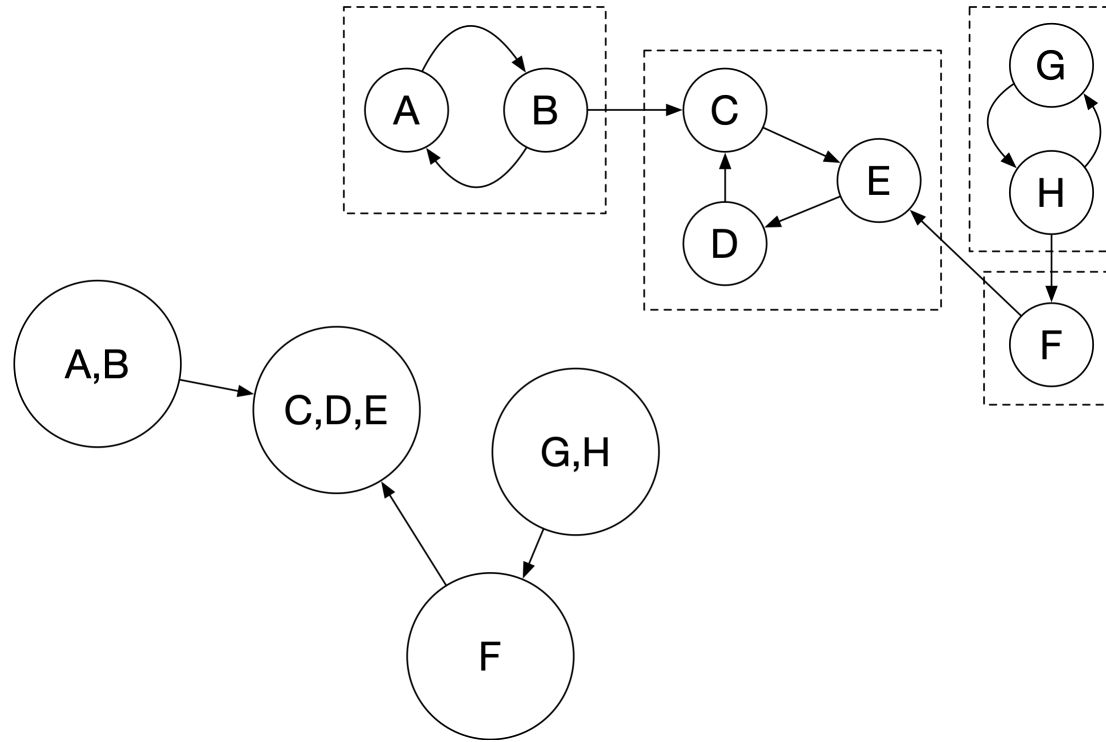
Strongly Connected Components

We can break a directed graph into its strongly connected components.



Strongly Connected Metagraph

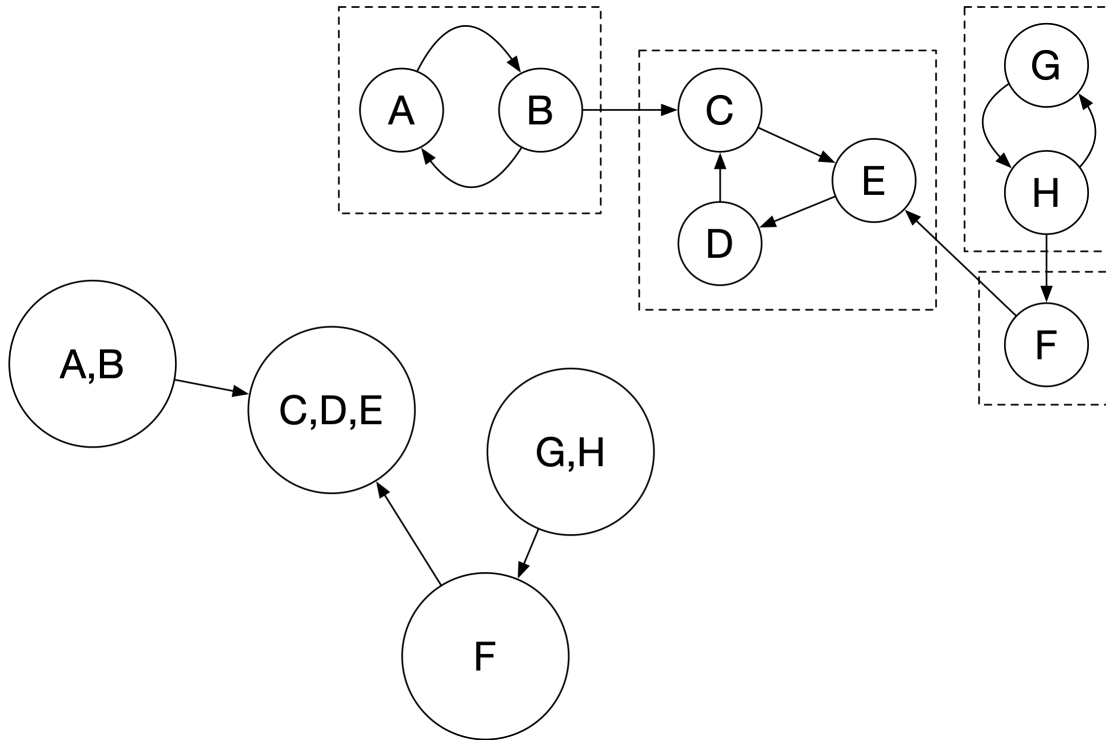
The result is a directed graph.



Anything special about the metagraph?

Strongly Connected Metagraph

The metagraph of strongly connected components is always a DAG

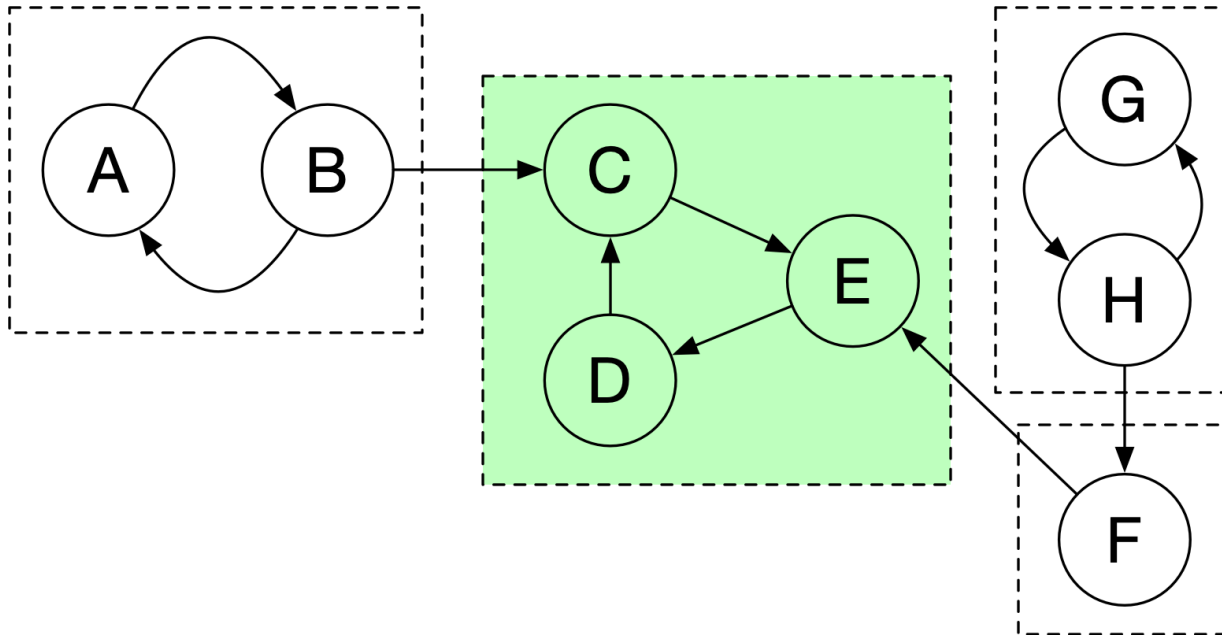


How do we know? Imagine if there were a cycle in the metagraph.

How can we identify the strongly connected components quickly?

A sink component

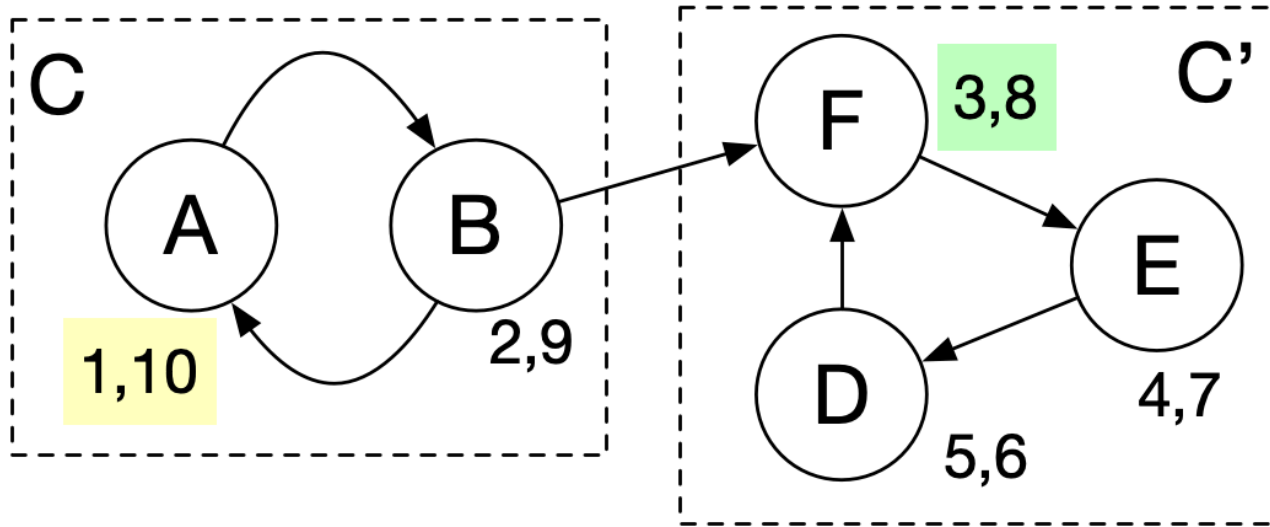
Explore on a sink component will explore exactly every vertex in the component.



How can we find the sink component?

Finding a source component

If C and C' are components with an edge from C to C' , then the highest post number in C is higher than the post number in C' .



Two cases: DFS starts in C (explore sees all of C') or DFS starts in C' (explore sees none of C).

The vertex with the highest post number *must* be in a source component.

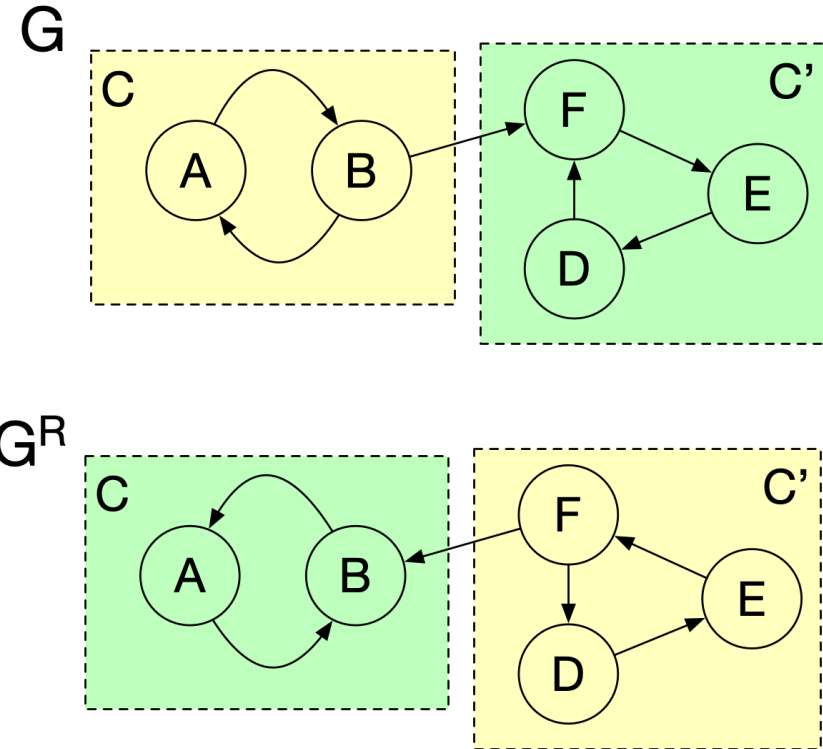
But...we need to find a node in a sink component.

Reverse Graph

Let G^R be the same as G , but with every edge reversed.

G^R and G have the same connected components.

However, a source component in G^R is a sink component in G .

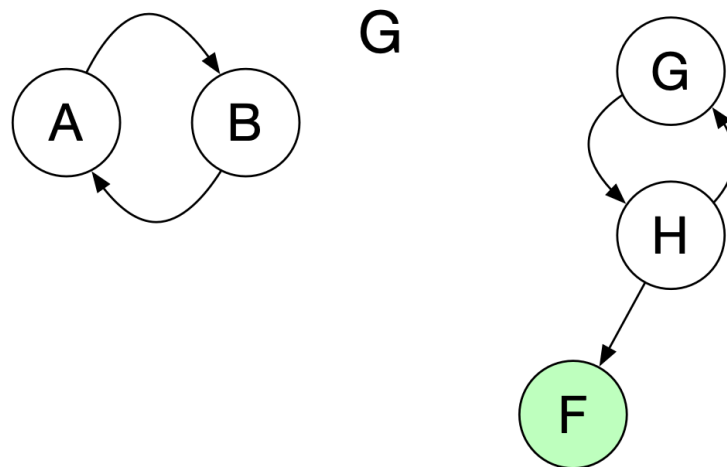
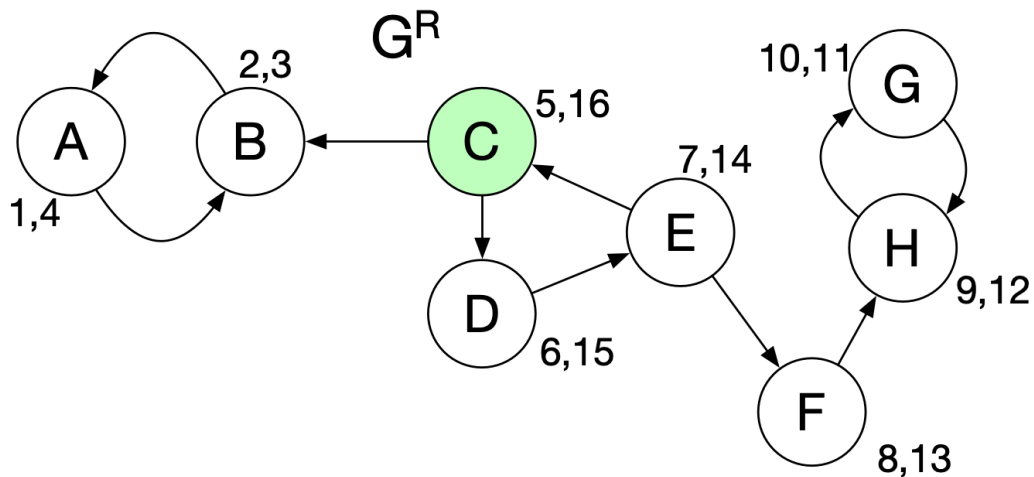
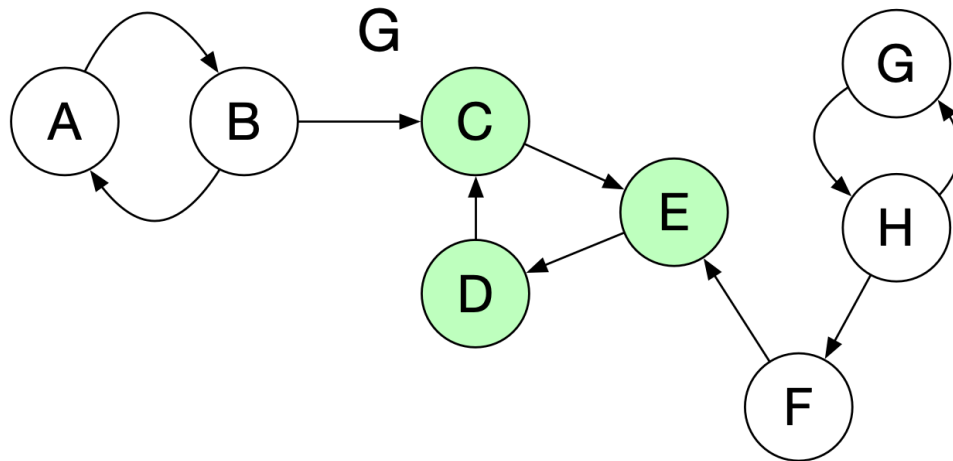
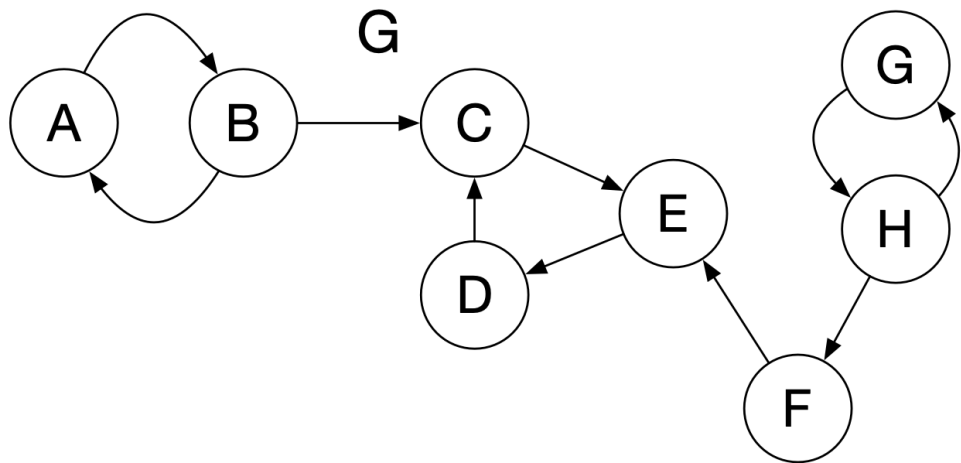


Identifying Components

- Make reverse graph G^R
- Run DFS on G^R and note post numbering
- While G has vertices:
 - Vertex with highest post value is in a sink component
 - Run Explore on that vertex in G to find vertices in sink component.
 - Delete components of current sink component

Identifying Components

27/28



Questions?

Slides by Aaron Hillegass